- Installed **Java 11** (compatible with **Hadoop 3.3.6**).
- Configured **JAVA_HOME** and updated **PATH** so Hadoop can locate the Java runtime.
- Enabled **passwordless SSH to localhost** by installing the SSH server and setting up **key-based authentication**.
- This is required because Hadoop start scripts (e.g., **start-dfs.sh / start-yarn.sh**) use SSH to launch daemons.
- Downloaded and installed **Hadoop 3.3.6**, then configured **HADOOP_HOME** and Hadoop environment variables.

-
```
rukhsaar@Rukhsaar:/usr/lib/jvm$ cd ~
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop

echo 'export HADOOP_HOME=$HOME/hadoop' >> ~/.bashrc
echo 'export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin' >> ~/.bashrc
source ~/.bashrc
hadoop version
--2026-02-17 10:30:49--  https://downloads.apache.org/hadoop/common/hadoop-3.3.6/
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 135.181.2
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... c
HTTP request sent, awaiting response... 200 OK
Length: 730107476 (696M) [application/x-gzip]
Saving to: 'hadoop-3.3.6.tar.gz'

hadoop-3.3.6.tar.gz                                 52%[=======================
```
- Configured HDFS settings in **core-site.xml** and **hdfs-site.xml**:
  - Edited: `nano ~/hadoop/etc/hadoop/core-site.xml`
  - Created HDFS storage dirs: `mkdir -p ~/hdfs/namenode ~/hdfs/datanode`
  - Edited: `nano ~/hadoop/etc/hadoop/hdfs-site.xml`
- Set key properties:
  - `fs.defaultFS = hdfs://localhost:9000` (core-site.xml)
  - NameNode/DataNode local storage directories (hdfs-site.xml)
  - `dfs.replication = 1` for single-node mode
- Formatted the NameNode **once** to initialize HDFS filesystem metadata.
- Started HDFS and verified daemons:
  - `start-dfs.sh`
  - `jps` to confirm NameNode/DataNode (and related HDFS services) are running.

Created the required folder structure in HDFS (raw zone and curated zone).

```
rukhsaar@Rukhsaar:~$ hdfs dfs -ls /
rukhsaar@Rukhsaar:~$ hdfs dfs -mkdir -p /loan_project/raw
rukhsaar@Rukhsaar:~$ hdfs dfs -mkdir -p /loan_project/curated
rukhsaar@Rukhsaar:~$ hdfs dfs -ls /loan_project
Found 2 items
drwxr-xr-x   - rukhsaar supergroup          0 2026-02-17 12:38 /loan_project/curated
drwxr-xr-x   - rukhsaar supergroup          0 2026-02-17 12:38 /loan_project/raw
rukhsaar@Rukhsaar:~$
```

Copy Dataset from Windows Path into Linux & Verified local file size.

```
rukhsaar@Rukhsaar:~$ cp "/mnt/c/OneDrive to disk/Ruk IOT/BDA Query/A2 - BDA2/raw csv/loan.csv" ~/loan.csv
rukhsaar@Rukhsaar:~$ ls -lh ~/loan.csv
-rwxr-xr-x 1 rukhsaar rukhsaar 1.2G Feb 17 12:50 /home/rukhsaar/loan.csv
rukhsaar@Rukhsaar:~$ nano /home/rukhsaar/loan.csv
rukhsaar@Rukhsaar:~$
```

Then Upload to HDFS Raw Zone (Uploaded raw dataset unchanged to HDFS raw zone.)

hdfs dfs -put -f ~/loan.csv /loan_project/raw/

hdfs dfs -ls -h /loan_project/raw

hdfs dfs -du -h /loan_project/raw

```
rukhsaar@Rukhsaar:~$ hdfs dfs -put -f ~/loan.csv /loan_project/raw/
rukhsaar@Rukhsaar:~$ hdfs dfs -ls -h /loan_project/raw
Found 1 items
-rw-r--r--   1 rukhsaar supergroup      1.1 G 2026-02-17 12:55 /loan_project/raw/loan.csv
rukhsaar@Rukhsaar:~$ hdfs dfs -du -h /loan_project/raw
1.1 G  1.1 G  /loan_project/raw/loan.csv
rukhsaar@Rukhsaar:~$
```

Row Count Proof (Reproducible)

Counted number of data records (excluding header line). This is evidence of dataset scale and successful HDFS storage/readability.

```
rukhsaar@Rukhsaar:~$ hdfs dfs -cat /loan_project/raw/loan.csv | tail -n +2 | wc -l
2260668
rukhsaar@Rukhsaar:~$
```

Reproducible Ingestion Script: nano ~/ingest_raw.sh

```
  GNU nano 7.2
#!/usr/bin/env bash
set -e

PROJECT=/loan_project
RAW_DIR=$PROJECT/raw
CURATED_DIR=$PROJECT/curated
LOCAL_CSV="$1"

if [ -z "$LOCAL_CSV" ]; then
  echo "Usage: $0 /path/to/loan.csv"
  exit 1
fi

hdfs dfs -mkdir -p "$RAW_DIR" "$CURATED_DIR"
hdfs dfs -put -f "$LOCAL_CSV" "$RAW_DIR/"

echo "=== HDFS raw listing ==="
hdfs dfs -ls -h "$RAW_DIR"
echo "=== HDFS raw size ==="
hdfs dfs -du -h "$RAW_DIR"
echo "=== Row count (excluding header) ==="
hdfs dfs -cat "$RAW_DIR/loan.csv" | tail -n +2 | wc -l
```

chmod +x ~/ingest_raw.sh

Then Validated the script performs all required steps automatically (upload + proof outputs), satisfying the “reproducible ingestion” requirement.

```
rukhsaar@Rukhsaar:~$ ~/ingest_raw.sh ~/loan.csv
=== HDFS raw listing ===
Found 1 items
-rw-r--r--   1 rukhsaar supergroup      1.1 G 2026-02-17 13:28 /loan_project/raw/loan.csv
=== HDFS raw size ===
1.1 G  1.1 G  /loan_project/raw/loan.csv
=== Row count (excluding header) ===
2260668
rukhsaar@Rukhsaar:~$
```

- Raw file exists in HDFS: `/loan_project/raw/loan.csv`
- Size proof: `1.1G`
- Row count (excluding header): `2,260,668`

Post-Ingestion Verification (Evidence)

```
rukhsaar@Rukhsaar:~$ jps
5860 Jps
4269 DataNode
4510 SecondaryNameNode
4143 NameNode
rukhsaar@Rukhsaar:~$ hdfs dfs -ls /loan_project
Found 2 items
drwxr-xr-x   - rukhsaar supergroup          0 2026-02-17 12:38 /loan_project/curated
drwxr-xr-x   - rukhsaar supergroup          0 2026-02-17 13:28 /loan_project/raw
rukhsaar@Rukhsaar:~$
```

This Confirmed HDFS is running and both raw and curated zones exist after ingestion.

GitHub Version Control (Clone, Add, Commit, Push)

Commit from your VM/WSL with git

Github token created: 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

```
rukhsaar@Rukhsaar:~$ cd ~
rukhsaar@Rukhsaar:~$ git clone https://github.com/Rukhss72/BDA6-LoanAnalytics-2026.git
Cloning into 'BDA6-LoanAnalytics-2026'...
Username for 'https://github.com': Rukhss72
Password for 'https://Rukhss72@github.com':
```

```
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 16 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (16/16), 6.45 KiB | 1.61 MiB/s, done.
Resolving deltas: 100% (3/3), done.
rukhsaar@Rukhsaar:~$
```

Commands:

git config --global user.name "Ruk"

git config --global user.email "baichoorukhsaar@gmail.com"

git commit -m "Add Ruk HDFS ingestion script + evidence"

git push

git status

Evidence Summary (What was produced)

- Hadoop installed and validated (`hadoop version`)
- HDFS configured, formatted, and running (`start-dfs.sh`, `jps`)
- HDFS zones created (`/loan_project/raw`, `/loan_project/curated`)
- Raw dataset stored in HDFS with size proof (`-ls -h`, `-du -h`)extraction complete
- Row count proof generated (`2260668`)
- Reproducible ingestion script created and tested (`ingest_raw.sh`)
- Code + documentation committed and pushed to GitHub (clean working tree)

NEXT = ETL Section

- **Why Pig (not Java MapReduce):**
  - **YARN** is Hadoop's resource manager; it allocates CPU/RAM and schedules jobs (including MapReduce) across nodes.
  - We chose **Apache Pig** because the timeline was short and Pig enables **faster ETL development** than writing and debugging Java MapReduce.
  - The dataset is **large and wide**, and the main tasks were **cleaning, normalization, and preparing outputs for KPIs**, which Pig handles well.
  - Pig scripts are automatically compiled into **MapReduce jobs** and executed on **YARN**, giving distributed processing with less code.
  - Result: a pipeline that is **scalable, efficient, and reproducible**.
- **ETL rules implemented in the Pig script (clean + normalize):**
  - Removed the **header row** from the curated output to prevent it becoming a fake record during aggregations.
  - **Normalized** `verification_status` (e.g., *Source Verified* → *Verified*).
  - Filled missing values for delinquency-related fields (e.g., `mths_since_last_delinq`, `mths_since_last_record`) with **0**.
  - Dropped **unused / high-null columns** (e.g., IDs, URL/description, hardship/debt settlement fields).
  - Stored the cleaned dataset into the **HDFS curated zone** (`/loan_project/curated/...`) for downstream querying and dashboards.

1. Start YARN

```
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
```

```
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$ jps
7104 ResourceManager
7239 NodeManager
4269 DataNode
4510 SecondaryNameNode
7599 Jps
4143 NameNode
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$
```

2. Configure Yarn via: nano ~/hadoop/etc/hadoop/mapred-site.xml
   nano ~/hadoop/etc/hadoop/yarn-site.xml

```
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$ cp ~/hadoop/etc/hadoop/mapred-site.xml.template ~/hadoop/etc/hadoop/mapred-site.xml
cp: cannot stat '/home/rukhsaar/hadoop/etc/hadoop/mapred-site.xml.template': No such file or directory
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$ ls -l ~/hadoop/etc/hadoop | grep mapred
-rw-r--r-- 1 rukhsaar rukhsaar   951 Jun 18  2023 mapred-env.cmd
-rw-r--r-- 1 rukhsaar rukhsaar  1764 Jun 18  2023 mapred-env.sh
-rw-r--r-- 1 rukhsaar rukhsaar  4113 Jun 18  2023 mapred-queues.xml.template
-rw-r--r-- 1 rukhsaar rukhsaar   758 Jun 18  2023 mapred-site.xml
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$ nano ~/hadoop/etc/hadoop/mapred-site.xml
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$
```

Download and install Apache Pig: Use Pig 0.17.0 (stable and common for Hadoop labs).

```
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026$ cd ~
rukhsaar@Rukhsaar:~$ wget https://archive.apache.org/dist/pig/pig-0.17.0/pig-0.17.0.tar.gz
--2026-02-17 18:39:04--  https://archive.apache.org/dist/pig/pig-0.17.0/pig-0.17.0.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 230606579 (220M) [application/x-gzip]
Saving to: 'pig-0.17.0.tar.gz'

pig-0.17.0.tar.gz                                   100%[=================================

2026-02-17 18:45:26 (593 KB/s) - 'pig-0.17.0.tar.gz' saved [230606579/230606579]

rukhsaar@Rukhsaar:~$ tar -xzf pig-0.17.0.tar.gz
rukhsaar@Rukhsaar:~$ mv pig-0.17.0 pig
rukhsaar@Rukhsaar:~$
```

Set environment variables = Add Pig to PATH:

```
rukhsaar@Rukhsaar:~$ echo 'export PIG_HOME=$HOME/pig' >> ~/.bashrc
rukhsaar@Rukhsaar:~$ echo 'export PATH=$PATH:$PIG_HOME/bin' >> ~/.bashrc
rukhsaar@Rukhsaar:~$ echo 'export PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop' >> ~/.bashrc
rukhsaar@Rukhsaar:~$ source ~/.bashrc
rukhsaar@Rukhsaar:~$ pig -version
Apache Pig version 0.17.0 (r1797386)
compiled Jun 02 2017, 15:41:58
rukhsaar@Rukhsaar:~$
```

Prepare the ETL script : RULES

1) Headers

- **RAW**: keep the header unchanged.
- **CURATED**: output **must not include** the header row.
- Optional (recommended): store a separate one-line header file in curated:
  - /loan_project/curated/loan_clean_header.csv

2) Normalize values

- verification_status: replace **"Source Verified" → "Verified"**

3) Default months-since fields

- If mths_since_last_delinq or mths_since_last_record is empty, set it to **0**

4) Drop low-value / mostly-empty columns

- Drop: id, member_id, pymnt_plan, url, desc, **all hardship_***, **all debt_settlement_***
- Also drop any other high-null fields not used in KPIs.

5) Handle remaining missing values

- Keep missing values as **typed NULL**, not the text "null"
  (Strings should also remain NULL where possible.)

Why remove the header row?

- The first row is: id,member_id,loan_amnt,...

- If you don't remove it, it becomes a **fake record** in the dataset.
- When Pig expects numeric types (e.g., loan_amnt:double, int_rate:double), header text can cause **cast/type issues** or incorrect aggregates.
- KPI results can break (e.g., grouping by loan_status may include "loan_status" as a category).
- Standard ETL practice: **raw keeps header, curated excludes header.**

Store a separate header file in curated (recommended if you need headers)

Keep curated output clean and store header separately:

- /loan_project/curated/loan_clean_header.csv (one line header)
- /loan_project/curated/loan_clean/part-* (data only)

Ex: echo "id,loan_amnt,term,int_rate,installment,grade,sub_grade,home_ownership,annual_inc,verification_status,issue_d,loan_status,purpose,addr_state,dti" > loan_clean_header.csv

hdfs dfs -put -f loan_clean_header.csv /loan_project/curated/loan_clean_header.csv

Add header only when exporting to BI tools

When exporting curated parts (e.g., `hdfs dfs -getmerge`), prepend the header locally before loading into Power BI/Excel.

Remove old outputs (safe even if they don't exist)

hdfs dfs -rm -r -f /loan_project/curated/loan_clean
hdfs dfs -rm -r -f /loan_project/curated/kpi_loan_status_counts

Run the ETL (MapReduce mode)

pig -x mapreduce ~/etl_loan.pig

Create + upload a separate header file for the curated dataset

```
cat > ~/loan_clean_header.csv << 'EOF'
id,loan_amnt,term,int_rate,installment,grade,sub_grade,home_ownership,annual_inc,verification
_status,issue_d,loan_status,purpose,addr_state,dti
EOF

hdfs dfs -put -f ~/loan_clean_header.csv /loan_project/curated/loan_clean_header.csv
```

Proof

```
hdfs dfs -ls -h /loan_project/curated
hdfs dfs -cat /loan_project/curated/loan_clean_header.csv
hdfs dfs -cat /loan_project/curated/loan_clean/part-* | head -n 3
hdfs dfs -cat /loan_project/curated/kpi_loan_status_counts/part-* | head -n 20
```

To avoid a broken script: create the Pig script with nano

1. Create the script:

```
nano ~/etl_loan_safe.pig
```

2. Paste the full script, then:
- Save: **Ctrl + O**, Enter
- Exit: **Ctrl + X**
3. Run cleanly:

```
hdfs dfs -rm -r -f /loan_project/curated/loan_clean
hdfs dfs -rm -r -f /loan_project/curated/kpi_loan_status_counts

pig -x mapreduce -stop_on_failure ~/etl_loan_safe.pig
```

4. Proof:

```
hdfs dfs -ls -h /loan_project/curated
hdfs dfs -cat /loan_project/curated/kpi_loan_status_counts/part-* | head -n 20
```

Search the log for the real error (copy/paste)

Run these:

```
grep -nEi "Caused by|line too long|RecordReader|Too long|InvalidInput|Exception|ERROR"
/home/rukhsaar/pig_1771342494018.log | tail -n 60
```

and also:

```
grep -nEi "Failed to read data|java\.io|org\.apache\.hadoop|Task failed|Attempt" /
```

Debug debug

Syntax errors etc..

Final debug code:

```
rukhsaar@Rukhsaar:~$

cat > ~/pig_read_only.pig << 'EOF'

SET mapreduce.input.linerecordreader.line.maxlength 50000000;

L = LOAD '/loan_project/raw/loan.csv' USING TextLoader() AS (line:chararray);

D = FILTER L BY line IS NOT NULL AND TRIM(line) != '' AND NOT STARTSWITH(line,
'id,');

S = LIMIT D 5;

DUMP S;

EOF

rukhsaar@Rukhsaar:~$ nl -ba ~/pig_read_only.pig 1 SET
mapreduce.input.linerecordreader.line.maxlength 50000000; 2 3 L = LOAD
'/loan_project/raw/loan.csv' USING TextLoader() AS (line:chararray); 4 D = FILTER L BY line
IS NOT NULL AND TRIM(line) != '' AND NOT STARTSWITH(line, 'id,'); 5 6 S = LIMIT D 5;
7 DUMP S;

rukhsaar@Rukhsaar:~$ pig -x mapreduce -stop_on_failure ~/pig_read_only.pig
```

2026-02-17 20:16:19,633 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL 2026-02-17 20:16:19,634 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE 2026-02-17 20:16:19,635 INFO pig.ExecTypeProvider: Picked .....

- Found out thatPig **parses fine**
- The job **runs**, but MapReduce fails with **"Failed to read data"** even for a simple `LOAD + FILTER + LIMIT`.
- That means the problem is **100% inside the raw CSV file format for Hadoop line-based reading**, not your Pig logic.

What's happening (most common with LendingClub-like loan.csv)

- Your CSV likely contains **embedded newlines inside quoted text fields** (usually `desc`), so Hadoop's `TextInputFormat` / `TextLoader` reads "lines" that are actually **broken fragments**. When a split starts mid-record, it can fail reading → Pig reports "Failed to read data".
- Even if you later "drop desc", it still breaks because the raw file itself is not line-safe.
- **Fix = sanitize the CSV into a "line-safe raw" copy**, then run Pig on that sanitized HDFS path.

Create a sanitized raw dataset (keeps original RAW untouched

Script to sanitize data:

```
python3 - << 'PY'

inp = "/home/rukhsaar/loan.csv"

out = "/home/rukhsaar/loan_sanitized.csv"


with open(inp, "r", encoding="utf-8", errors="replace") as f:

    header = f.readline().rstrip("\n")
```

```
expected_commas = header.count(",")
# we accept a row if it has at least 90% of header comma count
threshold = int(expected_commas * 0.9)

written = 0
buf = ""

with open(out, "w", encoding="utf-8") as g:
    g.write(header + "\n")

    for line in f:
        line = line.rstrip("\n")
        if not line.strip():
            continue

        c = line.count(",")

        if c >= threshold:
            # new "row-like" line
            if buf:
                g.write(buf + "\n")
                written += 1
            buf = line
        else:
            # continuation (embedded newline)
```

```
            if buf:
                buf += " " + line
            else:
                # ignore junk before first row
                continue

        if buf:
            g.write(buf + "\n")
            written += 1

print("Header commas:", expected_commas)
print("Row threshold:", threshold)
print("Sanitized records written:", written)
print("Output:", out)
PY
```

This shows a **real-world "raw data quality" issue**: multiline CSV records break Hadoop line readers → you created a **raw_sanitized** zone for processing while keeping **raw** unchanged for compliance/traceability.

Ran the improved sanitizer and paste the 4 printed numbers (header commas, threshold, records written, size).

Output:

Header commas: 144

Row threshold: 129

Sanitized records written: 2260668

Output: /home/rukhsaar/loan_sanitized.csv rukhsaar@Rukhsaar:~$ ls -lh ~/loan_sanitized.csv -rw-r--r-- 1

rukhsaar rukhsaar 1.2G Feb 17 20:19 /home/rukhsaar/loan_sanitized.csv
rukhsaar@Rukhsaar:~$ head -n 2 ~/loan_sanitized.csv
id,member_id,loan_amnt,funded_amnt.....

Upload sanitized to HDFS

```
hdfs dfs -mkdir -p /loan_project/raw_sanitized
hdfs dfs -put -f ~/loan_sanitized.csv
/loan_project/raw_sanitized/loan.csv
hdfs dfs -ls -h /loan_project/raw_sanitized
```

Re-test Pig read-only on sanitized

```
cat > ~/pig_read_only.pig << 'EOF'
SET mapreduce.input.linerecordreader.line.maxlength 50000000;

L = LOAD '/loan_project/raw_sanitized/loan.csv' USING TextLoader() AS
(line:chararray);
D = FILTER L BY line IS NOT NULL AND TRIM(line) != '' AND NOT
STARTSWITH(line, 'id,');

S = LIMIT D 5;
DUMP S;
EOF

Run pig
pig -x mapreduce -stop_on_failure ~/pig_read_only.pig
```

Then clean previous outputs (important)

```
hdfs dfs -rm -r -f /loan_project/curated/loan_clean
hdfs dfs -rm -r -f /loan_project/curated/kpi_loan_status_counts
```

Then run real ETL on sanitized raw

Create a NEW ETL script: `~/etl_loan_clean_only.pig`

This version:

- Loads sanitized CSV as raw lines (safe for huge/wide CSV)
- Skips header
- Splits by comma **only for the first 35 columns** (the ones you care about)
- Applies your cleaning rules
- Stores into curated zone

Confirm piggybank.jar path = find ~/pig -name "piggybank.jar" | head

Overwrite the Pig script properly (no "cat > …" inside)

Final script

```
cat > ~/etl_loan_clean_only.pig << 'EOF'

REGISTER /home/rukhsaar/pig/contrib/piggybank/java/piggybank.jar;

DEFINE CSVLoader org.apache.pig.piggybank.storage.CSVExcelStorage(',');


raw = LOAD '/loan_project/raw_sanitized/loan.csv' USING CSVLoader AS (

  id:chararray,

  member_id:chararray,

  loan_amnt:chararray,

  funded_amnt:chararray,

  funded_amnt_inv:chararray,

  term:chararray,
```

```
int_rate:chararray,
installment:chararray,
grade:chararray,
sub_grade:chararray,
emp_title:chararray,
emp_length:chararray,
home_ownership:chararray,
annual_inc:chararray,
verification_status:chararray,
issue_d:chararray,
loan_status:chararray,
pymnt_plan:chararray,
url:chararray,
descr:chararray,
purpose:chararray,
title:chararray,
zip_code:chararray,
addr_state:chararray,
dti:chararray,
delinq_2yrs:chararray,
earliest_cr_line:chararray,
inq_last_6mths:chararray,
mths_since_last_delinq:chararray,
mths_since_last_record:chararray,
open_acc:chararray,
```

```
  pub_rec:chararray,

  revol_bal:chararray,

  revol_util:chararray,

  total_acc:chararray

);


f = FILTER raw BY id != 'id';


clean = FOREACH f GENERATE

  loan_amnt,

  term,

  int_rate,

  installment,

  grade,

  sub_grade,

  emp_title,

  emp_length,

  home_ownership,

  annual_inc,

  (TRIM(verification_status) == 'Source Verified' ? 'Verified' : verification_status) AS
verification_status,

  issue_d,

  loan_status,

  purpose,

  title,
```

```
  addr_state,

  dti,

  delinq_2yrs,

  earliest_cr_line,

  inq_last_6mths,

  open_acc,

  pub_rec,

  revol_bal,

  revol_util,

  total_acc,

  ((mths_since_last_delinq IS NULL OR TRIM(mths_since_last_delinq) == '') ? '0' :
mths_since_last_delinq) AS mths_since_last_delinq,

  ((mths_since_last_record IS NULL OR TRIM(mths_since_last_record) == '') ? '0' :
mths_since_last_record) AS mths_since_last_record;



STORE clean INTO '/loan_project/curated/loan_clean_only' USING PigStorage(',');

EOF
```

Then run pig via: pig -x mapreduce etl_loan.pig

Clean output folder and run

```
hdfs dfs -rm -r -f /loan_project/curated/loan_clean_only
pig -x mapreduce -stop_on_failure ~/etl_loan_clean_only.pig
```

Verify it worked

```
hdfs dfs -test -e /loan_project/curated/loan_clean_only/_SUCCESS &&
echo OK || echo FAIL
hdfs dfs -cat /loan_project/curated/loan_clean_only/part-* | head -n 5
hdfs dfs -cat /loan_project/curated/loan_clean_only/part-* | wc -l
```

`wc -l` is > 0 = done

After export columns missmatched:

214	4 years	OWN	32000	Verified	Dec-15	Current	debt_consolidation	Debt consolidation	NE		26.50	1	Jun-79	2	5	0	10048	
215	10+ years	MORTGAGE	140000	Not Verified	Dec-15	Current	debt_consolidation	Debt consolidation	CO		5.04	0	Dec-89	0	13	0	6516	
216	< 1 year	RENT	40000	Verified	Dec-15	Current	debt_consolidation	Debt consolidation	CA		27.54	0	Jul-85	0	10	0	8671	
217	Mar	3 years	MORTGAGE	450000	Verified	Dec-15	Fully Paid	credit_card	Credit card refinancing	MA		12.37	0	Oct-97	0	21	0	36812
218	< 1 year	OWN	68000	Not Verified	Dec-15	Current	house	Home buying	FL			5.16	0	Aug-00	0	6	0	890
219	8 years	MORTGAGE	143000	Not Verified	Dec-15	Current	debt_consolidation	Debt consolidation	TX			24.53	0	Aug-87	0	16	0	44047
220	n/a	MORTGAGE	42000	Verified	Dec-15	Current	home_improvement	Home improvement	VA			12.17	2	Jul-87	1	5	0	5905
221	10+ years	MORTGAGE	75000	Not Verified	Dec-15	Current	debt_consolidation	Debt consolidation	MI			13.54	0	Oct-88	0	18	0	21322
222	Solutions Manu	9 years	RENT	280000	Verified	Dec-15	Current	debt_consolidation	Debt consolidation	IL			16.13	0	Jul-91	0	15	0	14
223	5 years	OWN	72000	Not Verified	Dec-15	Current	credit_card	Credit card refinancing	CA			11.42	0	Apr-88	1	9	0	8908
224	KEF	2 years	RENT	120000	Verified	Dec-15	Current	debt_consolidation	Debt consolidation	CA		5.89	0	Oct-14	0	7	0	15630
225	10+ years	MORTGAGE	68000	Verified	Dec-15	Current	debt_consolidation	Debt consolidation	SA			50.67	0	Nov-86	0	15	0	21426
226	7 years	RENT	40000	Verified	Dec-15	Current	credit_card	Credit card refinancing	MI			10.58	0	Oct-14	1	3	1	3092
227	3 years	MORTGAGE	49680	Not Verified	Dec-15	Current	credit_card	Credit card refinancing	NC			13.43	0	Jul-82	0	4	0	3257
228	5 years	RENT	66031	Not Verified	Dec-15	Current	debt_consolidation	Debt consolidation	NY			5.07	0	Sept-05	1	13	0	11136
229	n/a	RENT	13000	Not Verified	Dec-15	Current	credit_card	Credit card refinancing	PA			14.59	0	May-10	1	3	0	3931
230	5 years	MORTGAGE	37000	Verified	Dec-15	Current	debt_consolidation	Debt consolidation	TX			52.06	0	Oct-81	1	9	0	10606
231	5 years	MORTGAGE	30000	Verified	Dec-15	Current	debt_consolidation	Debt consolidation	NY			17.48	0	Oct-14	2	16	0	5323

Random Sampling = 100k rows from 2million

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\baich> wsl -d Ubuntu -u rukhsaar
rukhsaar@Rukhsaar:/mnt/c/Users/baich$ cd ~/BDA6-LoanAnalytics-2026/etl/scripts
chmod +x start_hadoop.sh
./start_hadoop.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Rukhsaar]
Starting resourcemanager
Starting nodemanagers
640 NameNode
1011 SecondaryNameNode
1574 Jps
763 DataNode
1373 NodeManager
1246 ResourceManager
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ hdfs dfs -ls /loan_project/curated/loan_clean_only | head
Found 10 items
-rwxrwxr-x   1 rukhsaar bda_team          0 2026-02-17 22:30 /loan_project/curated/loan_clean_only/_SUCCESS
-rwxrwxr-x   1 rukhsaar bda_team   45250536 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00000
-rwxrwxr-x   1 rukhsaar bda_team   45066396 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00001
-rwxrwxr-x   1 rukhsaar bda_team   45841072 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00002
-rwxrwxr-x   1 rukhsaar bda_team   46790358 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00003
-rwxrwxr-x   1 rukhsaar bda_team   48747817 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00004
-rwxrwxr-x   1 rukhsaar bda_team   46254699 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00005
-rwxrwxr-x   1 rukhsaar bda_team   44734109 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00006
-rwxrwxr-x   1 rukhsaar bda_team   46662965 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00007
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ hdfs dfs -cat /loan_project/curated/loan_clean_only/part-*
\
| awk 'BEGIN{srand()} rand() < 0.05 {print}' \
| head -n 100000 \
> ~/loan_clean_only_sample100k.csv
cat: Unable to write to output stream.
cat: Unable to write to output stream.
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ # header file you already created
HEADER=~/loan_clean_only_header.csv

hdfs dfs -cat /loan_project/curated/loan_clean_only/part-* \
| awk 'BEGIN{srand()} rand() < 0.05 {print}' \
| head -n 100000 \
| cat "$HEADER" - \
> ~/loan_clean_only_sample100k_with_header.csv

gzip -f ~/loan_clean_only_sample100k_with_header.csv
cat: Unable to write to output stream.
cat: Unable to write to output stream.
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ ls -lh ~/loan_clean_only_sample100k*.csv
-rw-r--r-- 1 rukhsaar rukhsaar 18M Feb 20 15:53 /home/rukhsaar/loan_clean_only_sample100k.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ wc -l  ~/loan_clean_only_sample100k.csv
100000 /home/rukhsaar/loan_clean_only_sample100k.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ wc -l  ~/loan_clean_only_sample100k_with_header.csv
wc: /home/rukhsaar/loan_clean_only_sample100k_with_header.csv: No such file or directory
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ head -n 2 ~/loan_clean_only_sample100k.csv
5550, 36 months,15.02,192.45,C,C3,Director COE,10+ years,MORTGAGE,152500,Not Verified,Dec-2018,Current,credit_card,C
redit card refinancing,IN,37.94,0,Sep-2002,3,18,0,53854,48.1,44,0,0
15500, 36 months,10.72,505.4,B,B2,Administrative Assistant ,10+ years,RENT,52000,Not Verified,Dec-2018,Current,credi
t_card,Credit card refinancing,CA,16.66,0,Oct-2007,0,13,0,13496,33.9,16,0,0
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ hdfs dfs -cat /loan_project/curated/loan_clean_only/part-*
2>/dev/null \
| awk 'BEGIN{srand()} rand()<0.05' \
| head -n 100000 \
> ~/loan_clean_only_sample100k.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ hdfs dfs -cat /loan_project/curated/loan_clean_only/part-*
2>/dev/null \
| awk 'BEGIN{srand(42)} rand()<0.05' \
| head -n 100000 \
> ~/loan_clean_only_sample100k.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ HEADER=~/loan_clean_only_header.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ cat "$HEADER" ~/loan_clean_only_sample100k.csv > ~/loan_cle
an_only_sample100k_with_header.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ head -n 2 ~/loan_clean_only_sample100k_with_header.csv
loan_amnt,term,int_rate,installment,grade,sub_grade,emp_title,emp_length,home_ownership,annual_inc,verification_stat
us,issue_d,loan_status,purpose,title,addr_state,dti,delinq_2yrs,earliest_cr_line,inq_last_6mths,open_acc,pub_rec,rev
ol_bal,revol_util,total_acc,mths_since_last_delinq,mths_since_last_record
6500, 36 months,17.97,234.9,D,D1,Quality Field Engineer,4 years,MORTGAGE,154000,Verified,Dec-2018,Current,debt_conso
lidation,Debt consolidation,NC,28.78,2,Oct-2008,0,27,0,23754,60.4,41,6,0
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ wc -l  ~/loan_clean_only_sample100k_with_header.csv
100001 /home/rukhsaar/loan_clean_only_sample100k_with_header.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ gzip -f ~/loan_clean_only_sample100k_with_header.csv
ls -lh ~/loan_clean_only_sample100k_with_header.csv.gz
-rw-r--r-- 1 rukhsaar rukhsaar 4.1M Feb 20 15:58 /home/rukhsaar/loan_clean_only_sample100k_with_header.csv.gz
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ cp -v ~/loan_clean_only_sample100k_with_header.csv.gz \
"/mnt/c/OneDrive to disk/Ruk IOT/BDA Query/A2 - BDA2/exported curated data from wsl/"
'/home/rukhsaar/loan_clean_only_sample100k_with_header.csv.gz' -> '/mnt/c/OneDrive to disk/Ruk IOT/BDA Query/A2 - BD
A2/exported curated data from wsl/loan_clean_only_sample100k_with_header.csv.gz'
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ ls -lh ~/loan_clean_only_header.csv && head -n 1 ~/loan_cle
an_only_header.csv
-rw-r--r-- 1 rukhsaar rukhsaar 306 Feb 18 19:45 /home/rukhsaar/loan_clean_only_header.csv
loan_amnt,term,int_rate,installment,grade,sub_grade,emp_title,emp_length,home_ownership,annual_inc,verification_stat
us,issue_d,loan_status,purpose,title,addr_state,dti,delinq_2yrs,earliest_cr_line,inq_last_6mths,open_acc,pub_rec,rev
ol_bal,revol_util,total_acc,mths_since_last_delinq,mths_since_last_record
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ |
```